

Design and Implementation of a Scalable Multi-User Database System for Smart Healthcare Management

Serhet Gökdemir
21058024

2026

Contents

1	Requirements Analysis	1
1.1	Introduction	1
1.2	User Roles and Permissions	1
1.3	Core Entities	1
1.4	Core Relationships	1
1.5	Constraints and Business Rules	2
2	ER and EER Models	2
2.1	ER Diagram	2
2.2	Enhanced ER (EER) Diagram	2
2.2.1	Explanation of EER Concepts	3
3	Mapping to Relational Model	3
3.1	Relational Schema	3
3.2	Mapping Rules Applied	8
4	Functional Dependencies and Normalization	8
4.1	Functional Dependencies (FDs)	8
4.2	Normalization Steps	11
5	SQL Implementation and Advanced Queries	15
5.1	Data Insertion	15
5.2	Advanced SQL Queries	15
5.2.1	Query Outputs	19
6	Relational Algebra & Calculus	19
7	Indexing and File Structures	21
7.1	Index Implementation	21
7.2	Performance Analysis	24
7.2.1	Indexed vs. Non-Indexed Query Performance	24

1 Requirements Analysis

1.1 Introduction

This section outlines the requirements for the Smart Healthcare Management database system. The primary objective is to create a robust, scalable, and secure system that supports multiple user roles and complex medical data interactions.

1.2 User Roles and Permissions

The system will support the following user roles:

- **Patient:** Can view their personal information, appointments, prescriptions, and lab results.
- **Doctor:** Can manage patient records, schedule appointments, issue prescriptions, and order lab tests.
- **Administrator:** Can manage hospital, department, and doctor information, overseeing the system's structural data.

1.3 Core Entities

The main entities identified for the system are:

- Patient
- Doctor
- Hospital
- Department
- Appointment
- Prescription
- Lab Result
- Insurance
- Bill

1.4 Core Relationships

Key relationships between entities include:

- A **Doctor** can have many **Patients** (1-to-N).
- A **Patient** can have many **Appointments** (1-to-N).
- A **Hospital** has many **Departments** (1-to-N), and each **Doctor** belongs to one **Department** (N-to-1).
- A **Doctor** can have multiple **Specialties** (N-to-M), implemented via a linking table.
- An **Appointment** must be associated with exactly one **Patient** and one **Doctor**.

1.5 Constraints and Business Rules

The system must enforce the following rules:

- A prescription can only be written by an authorized doctor.
- Every appointment must be linked to an existing patient and doctor.
- Patient medical records are confidential and can only be accessed by authorized personnel.

2 ER and EER Models

2.1 ER Diagram

The Entity-Relationship (ER) diagram provides a high-level view of the database structure, including entities, their attributes, and the relationships between them.

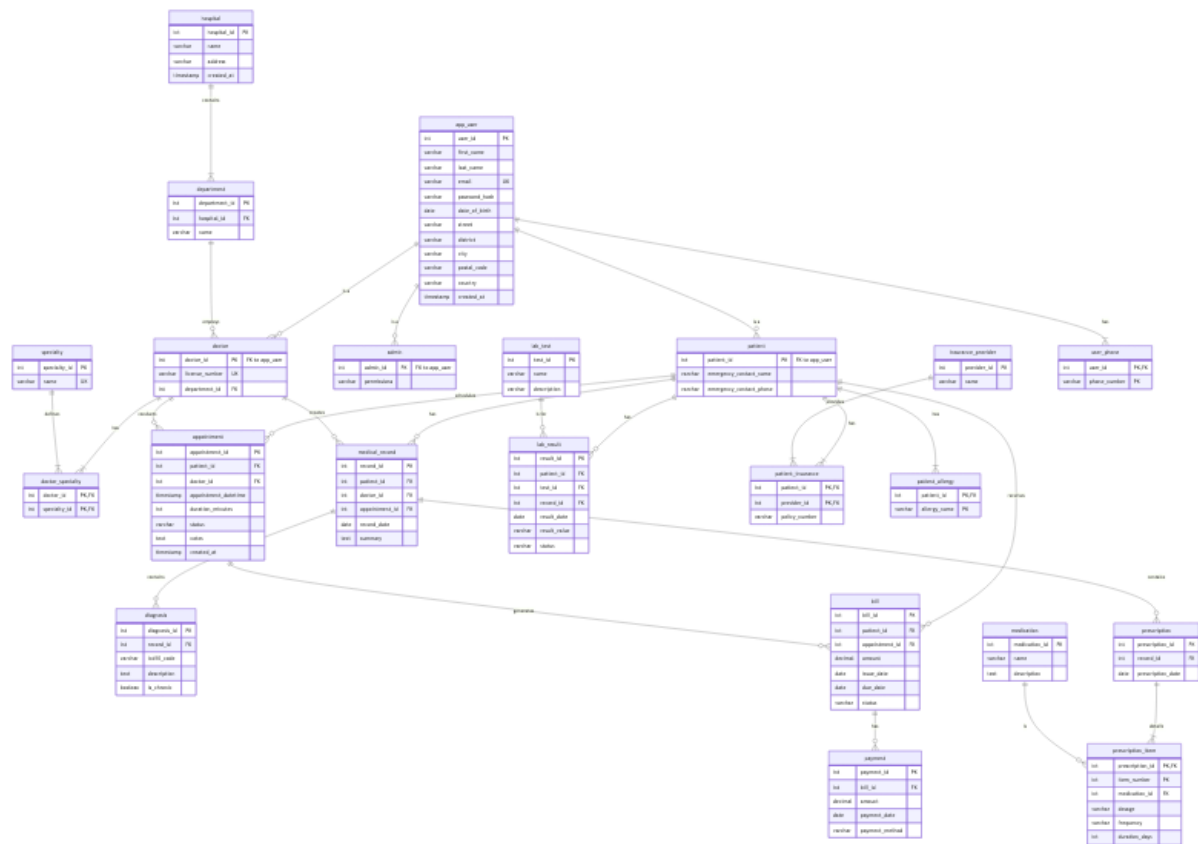


Figure 1: Entity-Relationship (ER) Diagram.

Note: The exported PNG is included in the repository. If you edit diagrams/er_diagram.mmd, re-export it to diagrams/er_diagram.png.

2.2 Enhanced ER (EER) Diagram

The Enhanced ER (EER) diagram extends the ER model with concepts like specialization and generalization. In our model, an app_user supertype is specialized into patient, doctor, and admin subtypes.

Note: The exported PNG is included in the repository. If you edit diagrams/er_diagram.mmd, re-export it to diagrams/er_diagram.png.

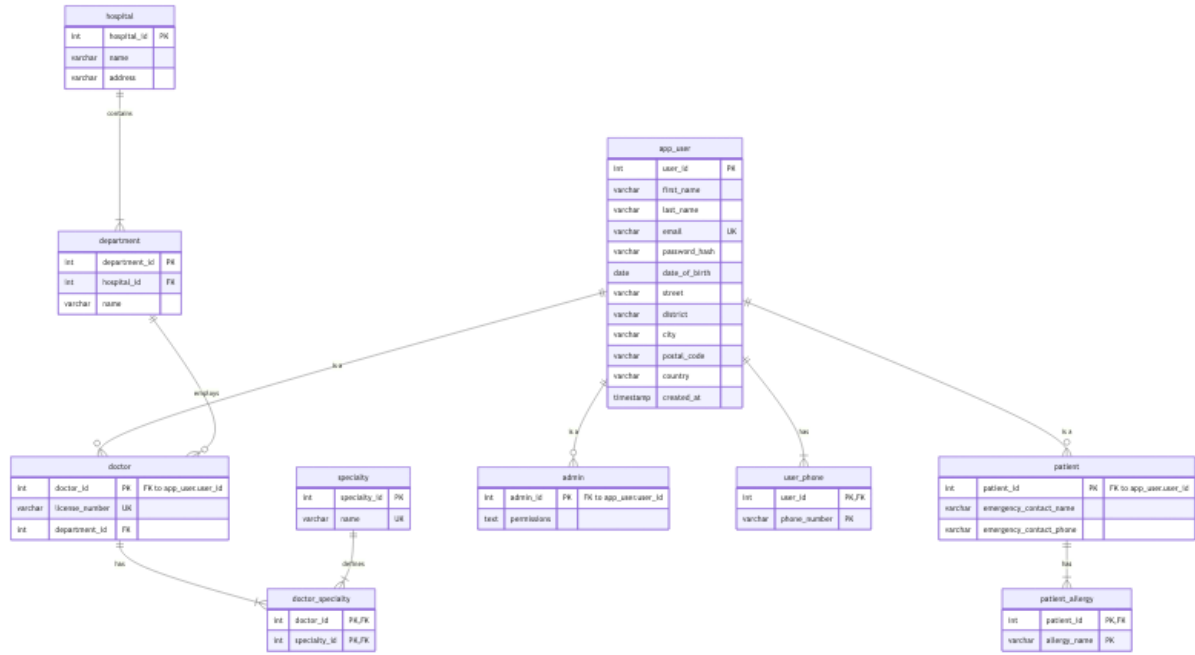


Figure 2: Enhanced Entity-Relationship (EER) Diagram.

2.2.1 Explanation of EER Concepts

Here, we used specialization to model different types of users. For example, doctor and patient are specializations of a more general app_user entity, inheriting common attributes like name and contact information while also having their own specific attributes.

3 Mapping to Relational Model

The ER/EER model was mapped to a relational schema. This section presents the final table structures, including primary keys, foreign keys, and other constraints.

3.1 Relational Schema

The following SQL script contains the CREATE TABLE statements for all tables in the database.

```

1  -- =====
2  -- Title: Smart Healthcare Management Database Schema
3  -- Description: PostgreSQL schema for a multi-user healthcare system.
4  -- =====
5
6  -- Drop existing tables to ensure a clean slate
7  DROP TABLE IF EXISTS
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 CASCADE;
16
17 -- Required for exclusion constraints using GiST with scalar equality
18 CREATE EXTENSION IF NOT EXISTS btree_gist;
  
```

```

16
17 -----
18 -- Hospitals and Departments
19 -----
20
21 CREATE TABLE hospital (
22     hospital_id SERIAL PRIMARY KEY,
23     name VARCHAR(100) NOT NULL,
24     address VARCHAR(255) NOT NULL,
25     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
26 );
27
28 CREATE TABLE department (
29     department_id SERIAL PRIMARY KEY,
30     hospital_id INT NOT NULL REFERENCES hospital(hospital_id) ON DELETE
31         CASCADE,
32     name VARCHAR(100) NOT NULL,
33     UNIQUE(hospital_id, name)
34 );
35 -----
36 -- User Management (Supertype-Subtype Structure)
37 -----
38
39 CREATE TABLE app_user (
40     user_id SERIAL PRIMARY KEY,
41     first_name VARCHAR(50) NOT NULL,
42     last_name VARCHAR(50) NOT NULL,
43     email VARCHAR(100) NOT NULL UNIQUE,
44     password_hash VARCHAR(255) NOT NULL,
45     date_of_birth DATE NOT NULL,
46     -- Composite attribute for address
47     street VARCHAR(100),
48     district VARCHAR(100),
49     city VARCHAR(50),
50     postal_code VARCHAR(20),
51     country VARCHAR(50),
52     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
53 );
54
55 CREATE TABLE patient (
56     patient_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
57         CASCADE,
58     emergency_contact_name VARCHAR(100),
59     emergency_contact_phone VARCHAR(20)
60 );
61
62 CREATE TABLE doctor (
63     doctor_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
64         CASCADE,
65     license_number VARCHAR(50) NOT NULL UNIQUE,
66     department_id INT REFERENCES department(department_id) ON DELETE SET
67         NULL
68 );
69
70 CREATE TABLE admin (
71     admin_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE
72     ,

```

```

69     permissions TEXT -- e.g., 'user_management,billing_access'
70 );
71
72 -----
73 -- Multivalued and N:M Relationship Tables
74 -----
75
76 -- Multivalued attribute for user phone numbers
77 CREATE TABLE user_phone (
78     user_id INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
79     phone_number VARCHAR(20) NOT NULL,
80     PRIMARY KEY (user_id, phone_number)
81 );
82
83 -- Multivalued attribute for patient allergies
84 CREATE TABLE patient_allergy (
85     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
86         CASCADE,
87     allergy_name VARCHAR(100) NOT NULL,
88     PRIMARY KEY (patient_id, allergy_name)
89 );
90
91 CREATE TABLE specialty (
92     specialty_id SERIAL PRIMARY KEY,
93     name VARCHAR(100) NOT NULL UNIQUE
94 );
95
96 -- N:M relationship between doctors and specialties
97 CREATE TABLE doctor_specialty (
98     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
99     specialty_id INT NOT NULL REFERENCES specialty(specialty_id) ON DELETE
100         CASCADE,
101     PRIMARY KEY (doctor_id, specialty_id)
102 );
103
104 -----
105 -- Core Healthcare Entities
106 -----
107
108 CREATE TABLE appointment (
109     appointment_id SERIAL PRIMARY KEY,
110     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
111         CASCADE,
112     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
113     appointment_datetime TIMESTAMP NOT NULL,
114     duration_minutes INT NOT NULL DEFAULT 30 CHECK (duration_minutes > 0),
115     status VARCHAR(20) NOT NULL CHECK (status IN ('scheduled', 'completed',
116         'cancelled', 'no-show')),
117     notes TEXT,
118     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
119     UNIQUE (appointment_id, patient_id)
120 );
121
122 -- Prevent a doctor from having overlapping scheduled/completed
123 -- appointments.
124 -- Note: This is stricter than enforcing uniqueness only at the exact start
125 -- time,
126 -- and it aligns with status logic by applying only to scheduled/completed

```

```

121 rows.
122 ALTER TABLE appointment
123 ADD CONSTRAINT no_doctor_overlap
124 EXCLUDE USING gist (
125     doctor_id WITH =,
126     tsrange(
127         appointment_datetime,
128         appointment_datetime + duration_minutes * INTERVAL '1 minute'
129     ) WITH &&
130 WHERE (status IN ('scheduled', 'completed')));
131
132 -- Prevent a patient from having overlapping scheduled/completed
133     appointments.
134 ALTER TABLE appointment
135 ADD CONSTRAINT no_patient_overlap
136 EXCLUDE USING gist (
137     patient_id WITH =,
138     tsrange(
139         appointment_datetime,
140         appointment_datetime + duration_minutes * INTERVAL '1 minute'
141     ) WITH &&
142 WHERE (status IN ('scheduled', 'completed')));
143
144 CREATE TABLE medical_record (
145     record_id SERIAL PRIMARY KEY,
146     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
147         CASCADE,
148     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
149     appointment_id INT REFERENCES appointment(appointment_id) ON DELETE SET
150         NULL,
151     record_date DATE NOT NULL DEFAULT CURRENT_DATE,
152     summary TEXT,
153     UNIQUE (record_id, patient_id)
154 );
155
156 CREATE TABLE diagnosis (
157     diagnosis_id SERIAL PRIMARY KEY,
158     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
159         CASCADE,
160     icd10_code VARCHAR(10),
161     description TEXT NOT NULL,
162     is_chronic BOOLEAN DEFAULT FALSE
163 );
164
165 CREATE TABLE medication (
166     medication_id SERIAL PRIMARY KEY,
167     name VARCHAR(100) NOT NULL UNIQUE,
168     description TEXT
169 );
170
171 CREATE TABLE prescription (
172     prescription_id SERIAL PRIMARY KEY,
173     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
174         CASCADE,
175     prescription_date DATE NOT NULL DEFAULT CURRENT_DATE
176 );

```



```

173
174 -- Weak entity: prescription_item depends on prescription
175 CREATE TABLE prescription_item (
176     prescription_id INT NOT NULL REFERENCES prescription(prescription_id)
177         ON DELETE CASCADE,
178     item_number INT NOT NULL, -- Partial key
179     medication_id INT NOT NULL REFERENCES medication(medication_id) ON
180         DELETE RESTRICT,
181     dosage VARCHAR(50) NOT NULL,
182     frequency VARCHAR(50) NOT NULL,
183     duration_days INT,
184     PRIMARY KEY (prescription_id, item_number)
185 );
186
187 CREATE TABLE lab_test (
188     test_id SERIAL PRIMARY KEY,
189     name VARCHAR(100) NOT NULL UNIQUE,
190     description TEXT
191 );
192
193 CREATE TABLE lab_result (
194     result_id SERIAL PRIMARY KEY,
195     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
196         CASCADE,
197     test_id INT NOT NULL REFERENCES lab_test(test_id) ON DELETE RESTRICT,
198     record_id INT REFERENCES medical_record(record_id) ON DELETE SET NULL,
199     result_date DATE NOT NULL,
200     result_value VARCHAR(100) NOT NULL,
201     status VARCHAR(20) CHECK (status IN ('normal', 'abnormal', 'pending')),
202     FOREIGN KEY (record_id, patient_id) REFERENCES medical_record(record_id
203         , patient_id)
204 );
205
206 -- -----
207 -- Billing and Insurance
208 -- -----
209
210 CREATE TABLE insurance_provider (
211     provider_id SERIAL PRIMARY KEY,
212     name VARCHAR(100) NOT NULL UNIQUE
213 );
214
215 -- N:M relationship between patients and insurance providers
216 CREATE TABLE patient_insurance (
217     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
218         CASCADE,
219     provider_id INT NOT NULL REFERENCES insurance_provider(provider_id) ON
220         DELETE CASCADE,
221     policy_number VARCHAR(50) NOT NULL,
222     PRIMARY KEY (patient_id, provider_id)
223 );
224
225 CREATE TABLE bill (
226     bill_id SERIAL PRIMARY KEY,
227     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
228         CASCADE,
229     appointment_id INT UNIQUE REFERENCES appointment(appointment_id) ON
230         DELETE SET NULL,

```

```

223     amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
224     issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
225     due_date DATE NOT NULL,
226     status VARCHAR(20) NOT NULL CHECK (status IN ('unpaid', 'paid', '
        partially_paid', 'overdue')),
227     FOREIGN KEY (appointment_id, patient_id) REFERENCES appointment(
        appointment_id, patient_id)
228 );
229
230 CREATE TABLE payment (
231     payment_id SERIAL PRIMARY KEY,
232     bill_id INT NOT NULL REFERENCES bill(bill_id) ON DELETE CASCADE,
233     amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
234     payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
235     payment_method VARCHAR(50) CHECK (payment_method IN ('credit_card', '
        bank_transfer', 'cash', 'insurance'))
236 );

```

Listing 1: Relational Schema (sql/schema.sql)

3.2 Mapping Rules Applied

Key mapping decisions included:

- **N:M Relationships:** Many-to-many relationships were implemented using separate linking tables (e.g., `doctor_specialty` for Doctor–Specialty and `patient_insurance` for Patient–Insurance Provider).
- **Weak Entities:** `prescription_item` is modeled as a weak entity identified by its owning `prescription` plus a partial key (`item_number`).
- **Multivalued Attributes:** Multivalued attributes are handled via separate tables (e.g., `user_phone` and `patient_allergy`).
- **Time Representation:** `appointment_datetime` is stored as `TIMESTAMP` (without time zone), assuming appointments are scheduled in the hospital’s local time zone.

4 Functional Dependencies and Normalization

4.1 Functional Dependencies (FDs)

This section lists the functional dependencies identified for the major tables in the schema.

```

1  # Functional Dependencies (FDs) and Normal Forms
2
3  This document outlines the functional dependencies for key tables in the
4  database, determines their candidate keys, and analyzes their normal
5  form.
6
7  ---
8
9  ### 1. `app_user` Table
10
11 This table stores core information for all users in the system.
12
13 **Attributes:** `user_id`, `first_name`, `last_name`, `email`, `
    password_hash`, `date_of_birth`, `street`, `district`, `city`, `
    postal_code`, `country`

```

```

12
13 **Functional Dependencies:**
14 -   '{user_id} -> {first_name, last_name, email, password_hash,
15     -   The primary key 'user_id' uniquely determines all other attributes.
16 -   '{email} -> {user_id, first_name, last_name, password_hash,
17     -   The 'email' is a unique identifier for a user.
18
19 **Candidate Keys:**
20 -   '{user_id}'
21 -   '{email}'
22
23 **Normalization Analysis:**
24 -   The table is in **BCNF**.
25 -   **Reasoning:** The determinants are '{user_id}' and '{email}'. Both are
26     candidate keys. Since all determinants are candidate keys, the table
27     satisfies the BCNF condition.
28
29 ---
30
31 ### 2. 'doctor' Table
32
33 This table is a subtype of 'app_user' and stores doctor-specific
34 information.
35
36 **Attributes:** 'doctor_id', 'license_number', 'department_id'
37
38 **Functional Dependencies:**
39 -   '{doctor_id} -> {license_number, department_id}'
40     -   'doctor_id' is the primary key and is also a foreign key to '
41         app_user(user_id)'.
42 -   '{license_number} -> {doctor_id, department_id}'
43     -   The 'license_number' is a unique value assigned to each doctor.
44
45 **Candidate Keys:**
46 -   '{doctor_id}'
47 -   '{license_number}'
48
49 **Normalization Analysis:**
50 -   The table is in **BCNF**.
51 -   **Reasoning:** The determinants are '{doctor_id}' and '{license_number
52     }'. Both are candidate keys. The table is in BCNF.
53
54 ---
55
56 ### 3. 'appointment' Table
57
58 This table records appointments between patients and doctors.
59
60 **Attributes:** 'appointment_id', 'patient_id', 'doctor_id', '
61     appointment_datetime', 'duration_minutes', 'status', 'notes'
62
63 **Functional Dependencies:**
64 -   '{appointment_id} -> {patient_id, doctor_id, appointment_datetime,
65     duration_minutes, status, notes}'
66     -   The primary key 'appointment_id' determines all other attributes of
67         the appointment.

```

```

60
61 Note: The implemented schema enforces no overlap for scheduled/completed
    appointments using exclusion constraints over a time range (based on `
    appointment_datetime` and `duration_minutes`). This is a business rule
    constraint rather than a strict functional dependency that yields
    additional candidate keys.
62
63 **Candidate Keys:**
64 -   `{appointment_id}` (Primary Key)
65
66 **Normalization Analysis:**
67 -   The table is in **BCNF**.
68 -   **Reasoning:** Assuming `appointment_id` is the sole primary key, it is
    the only determinant for the main FD. Other potential FDs rely on
    determinants that are also candidate keys. There are no transitive
    dependencies (e.g., `patient_id` does not determine `doctor_id`).
    Therefore, the table is in BCNF.
69
70 ---
71
72 ### 4. `prescription_item` Table
73
74 This is a weak entity that details the medications within a single
    prescription.
75
76 **Attributes:** `prescription_id`, `item_number`, `medication_id`, `dosage`
    `, `frequency`, `duration_days`
77
78 **Functional Dependencies:**
79 -   `{prescription_id, item_number} -> {medication_id, dosage, frequency,
    duration_days}`
80 -   The composite key, consisting of the foreign key `prescription_id`
    and the partial key `item_number`, uniquely determines the details
    of that specific line item in the prescription.
81
82 **Candidate Keys:**
83 -   `{prescription_id, item_number}`
84
85 **Normalization Analysis:**
86 -   The table is in **BCNF**.
87 -   **Reasoning:** The only determinant is the composite primary key `{
    prescription_id, item_number}`. Since this is a superkey, the table is
    in BCNF. All non-key attributes (`medication_id`, `dosage`, etc.) are
    fully dependent on the entire primary key, satisfying 2NF, and there are
    no transitive dependencies, satisfying 3NF and BCNF.
88
89 ---
90
91 ### 5. `bill` Table
92
93 This table stores billing information related to appointments or other
    services.
94
95 **Attributes:** `bill_id`, `patient_id`, `appointment_id`, `amount`, `
    issue_date`, `due_date`, `status`
96
97 **Functional Dependencies:**
98 -   `{bill_id} -> {patient_id, appointment_id, amount, issue_date, due_date`

```

```

    , status}`
99     - The primary key 'bill_id' determines all other attributes.
100
101 Note: The only full candidate key of the bill table is '{bill_id}'. For non
    -null appointment-linked bills, 'appointment_id' is unique and acts as a
    conditional determinant, but it is not treated as a full candidate key
    because it is nullable.
102
103 **Candidate Keys:**
104 - '{bill_id}'
105
106 Note: 'appointment_id' is UNIQUE when present, but since it is nullable it
    is not treated as a full candidate key for the entire table.
107
108 **Normalization Analysis:**
109 - The table is in **BCNF**.
110 - **Reasoning:** The only non-trivial functional dependency is determined
    by '{bill_id}', which is a candidate key. Therefore, the table
    satisfies BCNF.

```

Listing 2: Functional Dependencies (analysis/fd.md)

4.2 Normalization Steps

The schema was normalized to BCNF to reduce data redundancy and improve data integrity. The process is described below.

```

1 # Normalization of Healthcare Data
2
3 This document explains the process of normalizing a complex, unnormalized
4 table into a set of tables in Boyce-Codd Normal Form (BCNF).
5
6 ## Unnormalized Form (UNF)
7
8 Let's start with an intentionally unnormalized table, `
9 appointment_full_info`. This single table contains repeating groups and
10 transitive dependencies, making it inefficient and prone to anomalies.
11
12 **`appointment_full_info`**
13
14 | appointment_id | appointment_datetime | patient_id | patient_first_name |
15 | patient_last_name | patient_dob | doctor_id | doctor_first_name |
16 | doctor_last_name | doctor_license | department_id | department_name |
17 | hospital_id | hospital_name | diagnosis_codes | prescription_meds |
18 | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
19 | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
20 | 101 | 2026-06-10 10:00 | 1 | John | Smith | 1985-02-10 | 6 | Emily |
21 | Davis | LIC-CARD-1122 | 1 | Cardiology | 1 | City General | "I25.10, Z00
22 | .00" | "Atorvastatin 20mg, Lisinopril 10mg" |
23 | 102 | 2026-06-10 11:00 | 2 | Jane | Doe | 1990-07-22 | 7 | Michael |
24 | Miller | LIC-NEUR-3344 | 2 | Neurology | 1 | City General | "G43.909" |
25 | "Sumatriptan 50mg" |
26
27 **Problems with UNF:**
28 - **Repeating Groups:** `diagnosis_codes` and `prescription_meds` contain
29 multiple values in a single string. This violates 1NF.
30 - **Redundancy:** Patient details (name, DOB), doctor details, and
31 department/hospital info are repeated for every appointment.

```

```

19 - **Anomalies:**
20   - **Update Anomaly:** If Dr. Davis changes departments, we must update
    every record for her.
21   - **Insertion Anomaly:** We can't add a new doctor until they have an
    appointment. We can't add a new hospital unless it has a department
    with a doctor who has an appointment.
22   - **Deletion Anomaly:** If we delete John Smith's only appointment, we
    lose all information about him.
23
24 ---
25
26 ## First Normal Form (1NF)
27
28 **Rule:** Ensure all attributes are atomic and there are no repeating
    groups. Each cell should hold a single value.
29
30 We eliminate the repeating groups by creating separate tables for diagnoses
    and prescriptions related to a visit. In the implemented schema,
    diagnoses and prescriptions attach to a patient's
31 medical record (and prescriptions further decompose into line items).
32
33 The appointment_id-based diagnosis and prescription tables shown below are
    conceptual intermediate examples. In the final implemented schema,
    clinical data is attached to 'medical_record'.
34
35 **Decomposition:**
36 1. Create a diagnosis table to hold diagnoses for each visit/record.
37 2. Create a prescription table (and line items) to hold medications for
    each visit/record.
38
39 The main table now looks like this:
40
41 **'appointment_info_1nf'**
42
43 | appointment_id (PK) | appointment_datetime | patient_id |
    patient_first_name | ... | hospital_name |
44 | :--- | :--- | :--- | :--- | :--- | :--- |
45 | 101 | 2026-06-10 10:00 | 1 | John | ... | City General |
46 | 102 | 2026-06-10 11:00 | 2 | Jane | ... | City General |
47
48 **(Conceptual) Diagnosis table**
49
50 | appointment_id (FK) | diagnosis_code |
51 | :--- | :--- |
52 | 101 | I25.10 |
53 | 101 | Z00.00 |
54 | 102 | G43.909 |
55
56 **(Conceptual) Prescription table**
57
58 | appointment_id (FK) | medication_name | dosage |
59 | :--- | :--- | :--- |
60 | 101 | Atorvastatin | 20mg |
61 | 101 | Lisinopril | 10mg |
62 | 102 | Sumatriptan | 50mg |
63
64 **Improvement:** The data is now atomic, making it easier to query
    individual diagnoses or medications. However, massive redundancy still

```

```

exists.
65
66 ---
67
68 ## Second Normal Form (2NF)
69
70 **Rule:** Must be in 1NF, and all non-key attributes must be fully
    functionally dependent on the entire primary key. (This rule is most
    relevant for tables with composite primary keys).
71
72 Our 'appointment_info_1nf' table has a single-column primary key ('
    appointment_id'), so it is trivially in 2NF. However, it suffers from
    transitive dependencies, which are addressed by 3NF. Let's identify the
    functional dependencies to prepare for 3NF.
73
74 **Functional Dependencies in 'appointment_info_1nf':**
75 - 'appointment_id' -> 'appointment_datetime', 'patient_id', 'doctor_id'
76 - 'patient_id' -> 'patient_first_name', 'patient_last_name', 'patient_dob'
    (Partial Dependency if PK was composite)
77 - 'doctor_id' -> 'doctor_first_name', 'doctor_last_name', 'doctor_license',
    'department_id'
78 - 'department_id' -> 'department_name', 'hospital_id'
79 - 'hospital_id' -> 'hospital_name'
80
81 The dependencies on 'patient_id', 'doctor_id', etc., are **transitive
    dependencies**.
82
83 ---
84
85 ## Third Normal Form (3NF)
86
87 **Rule:** Must be in 2NF, and there should be no transitive dependencies. A
    non-key attribute cannot depend on another non-key attribute.
88
89 We decompose the table to eliminate these transitive dependencies.
90
91 **Decomposition:**
92 1. Create a 'patient' table.
93 2. Create a 'doctor' table.
94 3. Create a 'department' table.
95 4. Create a 'hospital' table.
96 5. The 'appointment' table will only contain attributes directly related
    to the appointment event itself, with foreign keys to the other tables.
97
98 **Final Normalized Tables (in 3NF):**
99
100 Note: The final implementation uses a supertype/subtype structure. Common
    person attributes (name, email, address, etc.) are stored in 'app_user',
    while 'patient', 'doctor', and 'admin' store role-specific attributes
    and reference 'app_user(user_id)'.
101
102 **'hospital'**
103 - 'hospital_id' (PK)
104 - 'name'
105 - FD: '{hospital_id} -> {name}'
106
107 **'department'**
108 - 'department_id' (PK)

```

```

109 - 'name'
110 - 'hospital_id' (FK)
111 - FD: '{department_id} -> {name, hospital_id}'
112
113 **'doctor'**
114 - 'doctor_id' (PK, also FK to 'app_user.user_id')
115 - 'license_number' (Candidate Key)
116 - 'department_id' (FK)
117 - FDs: '{doctor_id} -> {license_number, department_id}', '{license_number}
      -> {doctor_id, department_id}'
118
119 **'patient'**
120 - 'patient_id' (PK, also FK to 'app_user.user_id')
121 - (Role-specific attributes only; personal attributes live in 'app_user')
122 - FD: '{patient_id} -> {all patient-specific attributes}'
123
124 **'appointment'**
125 - 'appointment_id' (PK)
126 - 'appointment_datetime'
127 - 'patient_id' (FK)
128 - 'doctor_id' (FK)
129 - FD: '{appointment_id} -> {appointment_datetime, patient_id, doctor_id}'
130
131 This structure is now in 3NF. We have eliminated the redundancy and the
      update/insertion/deletion anomalies.
132
133 ---
134
135 ## Boyce-Codd Normal Form (BCNF)
136
137 **Rule:** For every non-trivial functional dependency 'X -> Y', 'X' must be
      a superkey.
138
139 BCNF is a stricter version of 3NF. A table is in BCNF if and only if every
      determinant is a candidate key.
140
141 Let's analyze our 3NF tables:
142 - **'hospital'**: The only determinant is 'hospital_id', which is the
      primary key (and thus a superkey). **In BCNF.**
143 - **'department'**: The only determinant is 'department_id', which is the
      primary key. **In BCNF.**
144 - **'patient'**: The only determinant is 'patient_id', which is the primary
      key. **In BCNF.**
145 - **'appointment'**: The only determinant is 'appointment_id', which is the
      primary key. **In BCNF.**
146 - **'doctor'**: We have two FDs:
147     1. '{doctor_id} -> {license_number, department_id}'
148     2. '{license_number} -> {doctor_id, department_id}'
149
150     The determinants are '{doctor_id}' and '{license_number}'. Both are
      candidate keys for the 'doctor' table. Since every determinant is a
      candidate key, the 'doctor' table **is in BCNF.**
151
152 **Conclusion:** The final decomposed schema ('hospital', 'department', '
      doctor', 'patient', 'appointment', etc.) is in BCNF, providing a robust,
      scalable, and anomaly-free design. The schema implemented in 'sql/
      schema.sql' follows this normalized structure.

```


5 SQL Implementation and Advanced Queries

5.1 Data Insertion

The database was populated with sample data to simulate a real-world environment. The script below shows a few example INSERT statements.

```

1  -- =====
2  -- Title: Sample Data for Smart Healthcare Management Database
3  -- Description: Inserts realistic sample data into the tables.
4  -- =====
5
6  -- Clear existing data
7  TRUNCATE
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 RESTART IDENTITY CASCADE;
16
17 -- -----
18 -- Hospitals and Departments

```

Listing 4: Sample Data Insertion (sql/data.sql)

5.2 Advanced SQL Queries

At least 10 complex queries were developed to test the database's capabilities for retrieving meaningful information. These queries include joins, aggregations, and subqueries.

```

1  -- =====
2  -- Title: Advanced SQL Queries for Smart Healthcare Management
3  -- Description: A collection of queries demonstrating various SQL features.
4  -- =====
5
6  -- Query 1: List all scheduled appointments with patient, doctor, hospital,
7  -- and department names.
8  -- Demonstrates: INNER JOIN on multiple tables.
9  SELECT
10     a.appointment_datetime,
11     p_user.first_name AS patient_first_name,
12     p_user.last_name AS patient_last_name,
13     d_user.first_name AS doctor_first_name,
14     d_user.last_name AS doctor_last_name,
15     h.name AS hospital_name,
16     dpt.name AS department_name
17 FROM appointment a
18 JOIN patient p ON a.patient_id = p.patient_id
19 JOIN app_user p_user ON p.patient_id = p_user.user_id
20 JOIN doctor doc ON a.doctor_id = doc.doctor_id

```

```

20 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
21 JOIN department dpt ON doc.department_id = dpt.department_id
22 JOIN hospital h ON dpt.hospital_id = h.hospital_id
23 WHERE a.status = 'scheduled'
24 ORDER BY a.appointment_datetime;
25
26
27 -- Query 2: Find doctors who have more than 2 appointments (scheduled or
28 -- completed).
29 -- Demonstrates: GROUP BY, HAVING, COUNT, and subquery in WHERE IN.
30 SELECT
31     d_user.first_name,
32     d_user.last_name,
33     COUNT(a.appointment_id) AS appointment_count
34 FROM doctor doc
35 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
36 JOIN appointment a ON doc.doctor_id = a.doctor_id
37 WHERE a.status IN ('scheduled', 'completed')
38 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
39 HAVING COUNT(a.appointment_id) > 2
40 ORDER BY appointment_count DESC;
41
42 -- Query 3: List patients who have no appointments at all.
43 -- Demonstrates: LEFT JOIN and IS NULL to find non-matching records.
44 SELECT
45     p_user.user_id,
46     p_user.first_name,
47     p_user.last_name
48 FROM patient p
49 JOIN app_user p_user ON p.patient_id = p_user.user_id
50 LEFT JOIN appointment a ON p.patient_id = a.patient_id
51 WHERE a.appointment_id IS NULL;
52
53
54 -- Query 4: Find the most common diagnosis based on ICD-10 code.
55 -- Demonstrates: GROUP BY, COUNT, ORDER BY, LIMIT.
56 SELECT
57     icd10_code,
58     description,
59     COUNT(*) AS diagnosis_count
60 FROM diagnosis
61 WHERE icd10_code IS NOT NULL
62 GROUP BY icd10_code, description
63 ORDER BY diagnosis_count DESC
64 LIMIT 5;
65
66
67 -- Query 5: Calculate the total bill amount and total amount paid per
68 -- patient.
69 -- Demonstrates: Subqueries in SELECT, LEFT JOIN, COALESCE, SUM, GROUP BY.
70 SELECT
71     p_user.first_name,
72     p_user.last_name,
73     COALESCE(SUM(b.amount), 0) AS total_billed,
74     (SELECT COALESCE(SUM(p.amount), 0)
75      FROM payment p
76      JOIN bill b2 ON p.bill_id = b2.bill_id

```

```

76         WHERE b2.patient_id = p_user.user_id) AS total_paid
77 FROM patient pat
78 JOIN app_user p_user ON pat.patient_id = p_user.user_id
79 LEFT JOIN bill b ON pat.patient_id = b.patient_id
80 GROUP BY p_user.user_id, p_user.first_name, p_user.last_name
81 ORDER BY total_billed DESC;
82
83
84 -- Query 6: List all unpaid/partially paid/overdue bills that are past
      their due date.
85 -- Demonstrates: Filtering by date, multiple conditions in WHERE.
86 SELECT
87     b.bill_id,
88     p_user.first_name,
89     p_user.last_name,
90     b.amount,
91     b.due_date,
92     b.status
93 FROM bill b
94 JOIN app_user p_user ON b.patient_id = p_user.user_id
95 WHERE b.status IN ('unpaid', 'partially_paid', 'overdue')
96        AND b.due_date < DATE '2026-07-01'
97 ORDER BY b.due_date;
98
99
100 -- Query 7: Find medications that have been prescribed more than 3 times.
101 -- Demonstrates: Nested query, GROUP BY, HAVING.
102 SELECT
103     m.name,
104     m.description,
105     pi_counts.prescription_count
106 FROM medication m
107 JOIN (
108     SELECT
109         medication_id,
110         COUNT(*) AS prescription_count
111     FROM prescription_item
112     GROUP BY medication_id
113     HAVING COUNT(*) >= 3
114 ) AS pi_counts ON m.medication_id = pi_counts.medication_id
115 ORDER BY pi_counts.prescription_count DESC;
116
117
118 -- Query 8: Show all lab results for a specific patient (John Smith,
      user_id=1).
119 -- Demonstrates: JOIN, filtering by a specific ID.
120 SELECT
121     lr.result_date,
122     lt.name AS test_name,
123     lr.result_value,
124     lr.status
125 FROM lab_result lr
126 JOIN lab_test lt ON lr.test_id = lt.test_id
127 WHERE lr.patient_id = 1
128 ORDER BY lr.result_date DESC;
129
130
131 -- Query 9: Find doctors in the 'Cardiology' department at 'City General

```

```

132     Hospital' and list their specialties.
133 -- Demonstrates: Multiple JOINS, filtering by text fields.
134 SELECT
135     d_user.first_name,
136     d_user.last_name,
137     s.name AS specialty_name
138 FROM doctor doc
139 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
140 JOIN department dpt ON doc.department_id = dpt.department_id
141 JOIN hospital h ON dpt.hospital_id = h.hospital_id
142 JOIN doctor_specialty ds ON doc.doctor_id = ds.doctor_id
143 JOIN specialty s ON ds.specialty_id = s.specialty_id
144 WHERE h.name = 'City General Hospital'
145     AND dpt.name = 'Cardiology';
146
147 -- Query 10: Find patients who have appointments but have no lab results.
148 -- Demonstrates: NOT EXISTS, correlated subquery.
149 SELECT DISTINCT
150     p_user.first_name,
151     p_user.last_name
152 FROM patient pat
153 JOIN app_user p_user ON pat.patient_id = p_user.user_id
154 JOIN appointment a ON pat.patient_id = a.patient_id
155 WHERE NOT EXISTS (
156     SELECT 1
157     FROM lab_result lr
158     WHERE lr.patient_id = pat.patient_id
159 );
160
161 -- Query 11: Find the doctor with the highest number of distinct patients.
162 -- Demonstrates: COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT.
163 SELECT
164     d_user.first_name,
165     d_user.last_name,
166     COUNT(DISTINCT a.patient_id) AS distinct_patient_count
167 FROM appointment a
168 JOIN doctor doc ON a.doctor_id = doc.doctor_id
169 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
170 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
171 ORDER BY distinct_patient_count DESC
172 LIMIT 1;
173
174
175 -- Query 12: Find the average bill amount by department for completed
176 -- appointments.
177 -- Demonstrates: AVG, GROUP BY, JOIN across multiple tables.
178 SELECT
179     dpt.name AS department_name,
180     h.name AS hospital_name,
181     AVG(b.amount) AS average_bill_amount
182 FROM bill b
183 JOIN appointment a ON b.appointment_id = a.appointment_id
184 JOIN doctor doc ON a.doctor_id = doc.doctor_id
185 JOIN department dpt ON doc.department_id = dpt.department_id
186 JOIN hospital h ON dpt.hospital_id = h.hospital_id
187 WHERE a.status = 'completed'
188 GROUP BY dpt.name, h.name

```

```
188 ORDER BY average_bill_amount DESC;
```

Listing 5: Advanced Queries (sql/queries.sql)

5.2.1 Query Outputs

Optional: You can run the queries and paste selected outputs here (e.g., inside a verbatim environment or as a formatted table). Outputs are not required to understand the schema and query design.

Query 1: List scheduled appointments with patient/doctor/hospital/department.

```
-- (Optional output)
```

Query 3: List patients with no appointments.

```
-- (Optional output)
```

6 Relational Algebra & Calculus

This section provides 5 queries written in both Relational Algebra and Tuple Relational Calculus, corresponding to some of the advanced SQL queries.

```
1 # Relational Algebra and Tuple Relational Calculus Examples
2
3 This document provides 5 query examples written in natural language,
4   relational algebra, and tuple relational calculus.
5
6 ---
7
8 ### Query 1: Find the names of all patients who have a scheduled
9   appointment.
10
11 - **Relational Algebra:**
12   \Pi_{first_name, last_name}(
13     (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
14     \bowtie_{patient.patient_id = appointment.patient_id}
15     (\sigma_{status='scheduled'}(appointment))
16   )
17
18 - **Tuple Relational Calculus:**
19   '{ t | \exists u \in app_user, \exists p \in patient, \exists a \in
20     appointment (
21       'u.user_id = p.patient_id \wedge p.patient_id = a.patient_id \wedge a.
22         status = 'scheduled' \wedge
23         't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
24
25 ---
26
27 ### Query 2: Find the names of all doctors working in the 'Cardiology'
28   department.
29
30 - **Relational Algebra:**
31   \Pi_{first_name, last_name}(
32     (app_user \bowtie_{app_user.user_id = doctor.doctor_id} doctor)
33     \bowtie_{doctor.department_id = department.department_id}
34     (\sigma_{name='Cardiology'}(department))
35   )
```

```

32 - **Tuple Relational Calculus:**
33   '{ t | \exists u \in app_user, \exists d \in doctor, \exists dept \in
      department (
34     'u.user_id = d.doctor_id \wedge d.department_id = dept.department_id \
      wedge dept.name = 'Cardiology' \wedge
35     't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
36
37 ---
38
39 ### Query 3: Find the amount and due date of all unpaid bills.
40
41 - **Relational Algebra:**
42   \Pi_{amount, due_date}(\sigma_{status='unpaid'}(bill))
43
44 - **Tuple Relational Calculus:**
45   '{ t | \exists b \in bill (b.status = 'unpaid' \wedge t.amount = b.
      amount \wedge t.due_date = b.due_date) }'
46
47 ---
48
49 ### Query 4: Find the names of patients who have at least one 'abnormal'
      lab result.
50
51 - **Relational Algebra:**
52   \Pi_{first_name, last_name}(
53     (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
54     \bowtie_{patient.patient_id = lab_result.patient_id}
55     (\sigma_{status='abnormal'}(lab_result))
56   )
57
58 - **Tuple Relational Calculus:**
59   '{ t | \exists u \in app_user, \exists p \in patient, \exists lr \in
      lab_result (
60     'u.user_id = p.patient_id \wedge p.patient_id = lr.patient_id \wedge lr
      .status = 'abnormal' \wedge
61     't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
62
63 ---
64
65 ### Query 5: Find the names of all medications prescribed to the patient
      with 'patient_id = 1'.
66
67 - **Relational Algebra:**
68   \Pi_{name}(
69     medication \bowtie_{medication.medication_id = prescription_item.
      medication_id}
70     prescription_item \bowtie_{prescription_item.prescription_id =
      prescription.prescription_id}
71     prescription \bowtie_{prescription.record_id = medical_record.
      record_id}
72     (\sigma_{patient_id=1}(medical_record))
73   )
74
75 - **Tuple Relational Calculus:**
76   '{ t | \exists m \in medication, \exists pi \in prescription_item, \
      exists p \in prescription, \exists mr \in medical_record (
77     'm.medication_id = pi.medication_id \wedge pi.prescription_id = p.
      prescription_id \wedge p.record_id = mr.record_id \wedge mr.

```

```

    patient_id = 1 \wedge `
78  `t.name = m.name) } `

```

Listing 6: Relational Algebra and Tuple Relational Calculus
(analysis/relational_algebra_and_calculus.md)

7 Indexing and File Structures

7.1 Index Implementation

To optimize query performance, several indexes were created. The choices include B+ Tree indexes for range queries and Hash indexes for exact lookups where applicable.

```

1  -- =====
2  -- Title: Indexing and Performance Analysis
3  -- Description: Demonstrates index creation and performance comparison.
4  -- =====
5
6  -- Note:
7  -- Because the sample dataset is small, PostgreSQL may still choose a
8  -- sequential scan
9  -- even after an index is created. This is not an error; index benefits are
10 -- clearer
11 -- on larger datasets.
12
13 -- Clean up any existing indexes to ensure a clean run
14 DROP INDEX IF EXISTS idx_appointment_patient_id;
15 DROP INDEX IF EXISTS idx_appointment_doctor_datetime;
16 DROP INDEX IF EXISTS idx_user_email_hash;
17 DROP INDEX IF EXISTS idx_bill_patient_status;
18 DROP INDEX IF EXISTS idx_lab_result_patient_date;
19
20 -- -----
21 -- Example 1: B+ Tree Index on a Foreign Key
22 -- -----
23
24 -- Query: Find all appointments for a specific patient.
25 -- This is a very common operation in a healthcare system.
26
27 -- Before index:
28 EXPLAIN ANALYZE
29 SELECT * FROM appointment WHERE patient_id = 1;
30
31 -- Result (example): Seq Scan on appointment (cost=0.00..45.50 rows=5 width
32 -- =33) (actual time=...)
33 -- The database has to scan the entire 'appointment' table.
34
35 -- Create a B+ Tree index (the default type in PostgreSQL)
36 CREATE INDEX idx_appointment_patient_id ON appointment(patient_id);
37
38 -- After index:
39 EXPLAIN ANALYZE
40 SELECT * FROM appointment WHERE patient_id = 1;
41
42 -- Result (example): Index Scan using idx_appointment_patient_id on
43 -- appointment (cost=0.28..8.29 rows=5 width=33) (actual time=...)
44 -- The database now uses the index for a much faster lookup.

```

```

41  -- -----
42  -- Example 2: Composite B+ Tree Index
43  -- -----
44  -- Query: Find all appointments for a doctor within a specific date range.
45  -- This is useful for a doctor's calendar view.
46
47  -- Before index:
48  EXPLAIN ANALYZE
49  SELECT appointment_id, appointment_datetime, status
50  FROM appointment
51  WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
      appointment_datetime < '2026-07-01';
52
53  -- Result (example): Seq Scan on appointment... (filters on both doctor_id
      and datetime)
54
55  -- Create a composite index on doctor_id and appointment_datetime
56  CREATE INDEX idx_appointment_doctor_datetime ON appointment (doctor_id,
      appointment_datetime);
57
58  -- After index:
59  EXPLAIN ANALYZE
60  SELECT appointment_id, appointment_datetime, status
61  FROM appointment
62  WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
      appointment_datetime < '2026-07-01';
63
64  -- Result (example): Index Scan using idx_appointment_doctor_datetime...
65  -- The index allows efficient lookup of the doctor and then scanning the
      relevant date range within that doctor's appointments.
66
67  -- -----
68  -- Example 3: Hash Index for Exact Lookups
69  -- -----
70  -- Query: Find a user by their exact email address.
71  -- This is common during login or user registration checks. Hash indexes
      are optimized for equality checks.
72
73  -- Before index (assuming only the UNIQUE constraint's B-Tree index exists)
      :
74  EXPLAIN ANALYZE
75  SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
      doe@example.com';
76
77  -- Result (example): Index Scan using app_user_email_key... (B-Tree)
78
79  -- Better explanation:
80  -- PostgreSQL automatically creates a B-tree index for UNIQUE(email).
81  -- The following HASH index is included only to demonstrate hash index
      syntax.
82  -- In a real system, the UNIQUE B-tree index is usually sufficient.
83  CREATE INDEX idx_user_email_hash ON app_user USING HASH (email);
84
85  -- After index:
86  EXPLAIN ANALYZE
87  SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
      doe@example.com';
88

```



```

89 -- Result (example): Bitmap Heap Scan on app_user... -> Bitmap Index Scan
    on idx_user_email_hash...
90 -- The planner might choose the hash index for this equality-only query,
    which can be more memory-efficient and faster for very large tables.
91
92 -- -----
93 -- Example 4: Composite Index for Filtering and Sorting
94 -- -----
95 -- Query: Find unpaid bills for a patient, ordered by due date.
96 -- This helps in displaying a patient's outstanding bills in a logical
    order.
97
98 -- Before index:
99 EXPLAIN ANALYZE
100 SELECT bill_id, amount, due_date
101 FROM bill
102 WHERE patient_id = 3 AND status = 'partially_paid'
103 ORDER BY due_date;
104
105 -- Result (example): Seq Scan on bill... followed by a Sort operation.
    Sorting can be expensive.
106
107 -- Create a composite index to cover filtering and ordering
108 CREATE INDEX idx_bill_patient_status ON bill(patient_id, status, due_date);
109
110 -- After index:
111 EXPLAIN ANALYZE
112 SELECT bill_id, amount, due_date
113 FROM bill
114 WHERE patient_id = 3 AND status = 'partially_paid'
115 ORDER BY due_date;
116
117 -- Result (example): Index Scan using idx_bill_patient_status...
118 -- The database can use the index to find the matching rows and retrieve
    them in the pre-sorted order of 'due_date', avoiding a separate Sort
    step.
119
120 -- -----
121 -- Example 5: Index on Lab Results
122 -- -----
123 -- Query: Retrieve a patient's lab history, sorted by date.
124 -- A common query for reviewing a patient's medical history.
125
126 -- Before index:
127 EXPLAIN ANALYZE
128 SELECT result_id, test_id, result_date, result_value
129 FROM lab_result
130 WHERE patient_id = 1
131 ORDER BY result_date DESC;
132
133 -- Result (example): Seq Scan on lab_result... followed by a Sort.
134
135 -- Create a composite index
136 CREATE INDEX idx_lab_result_patient_date ON lab_result(patient_id,
    result_date DESC);
137
138 -- After index:
139 EXPLAIN ANALYZE

```

```

140 SELECT result_id, test_id, result_date, result_value
141 FROM lab_result
142 WHERE patient_id = 1
143 ORDER BY result_date DESC;
144
145 -- Result (example): Index Scan using idx_lab_result_patient_date...
146 -- The index provides fast access to the patient's results and delivers
   them in the correct descending order by date.

```

Listing 7: Index Creation (sql/indexing.sql)

7.2 Performance Analysis

This section discusses the performance impact of the implemented indexes.

7.2.1 Indexed vs. Non-Indexed Query Performance

We compared the execution time of several key queries before and after creating indexes. For example, finding all appointments for a specific patient, retrieving a doctor's appointments within a date range, or listing a patient's lab history sorted by date.

Example Query: Find all appointments for a specific patient.

- **Without Index:** The database may perform a sequential scan on the `appointment` table. See the `EXPLAIN ANALYZE` output for the plan and timing.
- **With B+ Tree Index on `appointment (patient_id)`:** The database can use an index scan to quickly locate matching rows. See the `EXPLAIN ANALYZE` output for the plan and timing.

On larger datasets, indexes are expected to improve query performance by reducing the amount of data scanned and by avoiding expensive sorts. With a small sample dataset, PostgreSQL may still choose sequential scans, which is not an error but a cost-based planner decision.